

LAB FILE

SCALA PROGRAMMING

Practical Experiments in Functional and Object-Oriented Programming

Name of Student:

Enrollment / Roll No.:

Course / Branch:

Semester / Section:

Submitted To:

Academic Session:

Name of Institute / Department

INDEX

S.No.	Name of Experiment	Page	Date	Sign.
1	Study of Basic Syntax, Variables and Data Types in Scala			
2	Implementation of Control Structures in Scala			
3	Functions in Scala			
4	Strings and Arrays in Scala			
5	Collections: List, Set, Map and Tuple			
6	Higher-Order Functions on Collections			
7	Classes, Objects and Case Classes			
8	Inheritance, Abstract Classes and Traits			
9	Pattern Matching, Option and Exception Handling			
10	File Handling and a Mini Application			

EXPERIMENT 1

Study of Basic Syntax, Variables and Data Types in Scala

Aim

To study the basic structure of a Scala program, variables, data types and console input/output.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 1

Write a Scala program that:

- (a) Prints Hello, World! using an object with a main method (also try the App trait).
- (b) Declares variables using `val` and `var`, and shows what happens when a `val` is reassigned.
- (c) Creates variables of types `Int`, `Long`, `Double`, `Float`, `Char`, `String` and `Boolean`, and prints each value.
- (d) Reads two numbers from the user using `scala.io.StdIn.readLine()` and performs arithmetic, relational and logical operations on them.
- (e) Uses string interpolation (`s""` and `f""`) to print a formatted result.
- (f) Demonstrates type inference and explicit type conversion (`toInt`, `toDouble`, `toString`).

Date: _____

EXPERIMENT 2

Implementation of Control Structures in Scala

Aim

To implement decision-making and looping constructs in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 2

Write Scala programs to:

- (a) Check whether a number is positive, negative or zero using `if...else if...else` (note that `if` returns a value).
- (b) Find the largest of three numbers.
- (c) Print the multiplication table of a given number using a `for` loop.
- (d) Print the numbers from 1 to 20 using `for` with `to`, `until`, a step (`by`) and a filter guard (`if`).
- (e) Compute the factorial of a number using a `while` loop and a `do...while` loop.
- (f) Print the Fibonacci series up to `n` terms.
- (g) Implement a simple menu-driven calculator using `match`.

Date: _____

EXPERIMENT 3

Functions in Scala

Aim

To define and use functions, including recursion and higher-order functions, in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 3

Write Scala programs to:

- (a) Define functions using `def` with parameters and return types, and call them using positional and named arguments.
- (b) Write a function with default parameter values and a function with variable arguments (*).
- (c) Write recursive functions to compute the factorial and the n-th Fibonacci number.
- (d) Write a tail-recursive version of the factorial using `@tailrec` and compare it with the normal recursion.
- (e) Define an anonymous function (lambda) and store it in a `val`.
- (f) Write a higher-order function that takes another function as an argument (for example, `applyTwice(f, x)`).
- (g) Demonstrate currying and a nested (local) function.

Date: _____

EXPERIMENT 4

Strings and Arrays in Scala

Aim

To perform common operations on strings and arrays in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 4

Write Scala programs to:

- (a) Find the length of a string, convert it to upper and lower case, and extract a substring.
- (b) Check whether a given string is a palindrome.
- (c) Count the vowels and consonants in a string.
- (d) Create an Array of 10 integers and find its sum, maximum, minimum and average.
- (e) Reverse and sort an array without modifying the original, and access elements by index.
- (f) Create a 2D array (3x3 matrix) and print it row by row; then find the sum of the diagonal elements.
- (g) Search for an element in an array using linear search.

Date: _____

EXPERIMENT 5

Collections: List, Set, Map and Tuple

Aim

To study and implement the basic Scala collections and their operations.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 5

Write Scala programs to:

- (a) Create an immutable `List` and use `head`, `tail`, `take`, `drop`, `reverse`, `:+`, `+:` and `:::`.
- (b) Create a `ListBuffer` or `ArrayBuffer` and add, update and remove elements.
- (c) Create two `Sets` and perform union, intersection and difference; show that a `Set` removes duplicates.
- (d) Create a `Map` of student names and marks; add, update, remove and look up entries, and iterate over the map.
- (e) Use `getOrElse` and `contains` for safe `Map` access.
- (f) Create tuples and access their elements (`_1`, `_2`); use `zip` to combine two lists into a list of tuples.
- (g) Compare mutable and immutable collections with a short example.

Date: _____

EXPERIMENT 6

Higher-Order Functions on Collections

Aim

To process collections using functional programming methods such as map, filter and fold.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 6

Given a list of integers `List(5, 12, 7, 20, 3, 18, 9, 14)`, write Scala programs to:

- (a) Use `map` to square every element and `filter` to keep only the even numbers.
- (b) Use `reduce`, `foldLeft` and `sum` to compute the sum and the product of the elements.
- (c) Use `flatMap` on a list of sentences to produce a list of all the words.
- (d) Use `exists`, `forall`, `find`, `count` and `partition` on the list.
- (e) Use `sortBy`, `sortWith`, `take`, `takeWhile` and `dropWhile`.
- (f) Use `groupBy` to group a list of words by their length, and then count the words in each group.
- (g) Use a for comprehension with `yield` to generate all pairs (i, j) where $i < j$, for values from 1 to 4.

Date: _____

EXPERIMENT 7

Classes, Objects and Case Classes

Aim

To implement classes, objects, constructors and case classes in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 7

Write Scala programs to:

- (a) Define a class `Student` with a primary constructor (`name`, `rollNo`, `marks`) and a method `grade()` that returns the grade.
- (b) Add an auxiliary constructor and a method that displays the student details.
- (c) Create a class `BankAccount` with a private balance and methods `deposit()`, `withdraw()` and `getBalance()`; reject invalid withdrawals.
- (d) Define a companion object with an `apply()` method and a factory method for the `Student` class.
- (e) Define a case class `Employee(name, dept, salary)`; demonstrate `copy()`, `toString`, `equality` and creating instances without `new`.
- (f) Create a singleton object holding utility functions (such as `max`, `min` and `isPrime`).
- (g) Store several `Employee` objects in a `List` and find the employee with the highest salary.

Date: _____

EXPERIMENT 8

Inheritance, Abstract Classes and Traits

Aim

To implement inheritance, polymorphism, abstract classes and traits in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 8

Write Scala programs to:

- (a) Create a base class `Person` and a derived class `Teacher` using `extends`; override a method using `override`.
- (b) Define an abstract class `Shape` with abstract methods `area()` and `perimeter()`, and implement it in `Circle`, `Rectangle` and `Triangle`.
- (c) Demonstrate runtime polymorphism by storing different shapes in a `List[Shape]` and printing the area of each.
- (d) Define a trait `Logger` with a concrete method and an abstract method, and mix it into a class using `extends` and `with`.
- (e) Demonstrate multiple trait inheritance and show the order in which the traits are linearized.
- (f) Create a sealed trait with case classes and a function that handles each case.

Date: _____

EXPERIMENT 9

Pattern Matching, Option and Exception Handling

Aim

To use pattern matching, the Option type and exception handling for safe programming in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 9

Write Scala programs to:

- (a) Use match with literal, variable, type and guard (if) patterns, and a default case _.
- (b) Pattern match on a List, a tuple and a case class.
- (c) Write a function that returns Option[Int] (for example, a safe division or a lookup in a Map) and handle Some and None using match, getOrElse and map.
- (d) Handle exceptions with try, catch and finally (for example, ArithmeticException and NumberFormatException).
- (e) Use scala.util.Try with Success and Failure to parse a list of strings into integers safely.
- (f) Throw a custom exception by extending Exception and handle it.

Date: _____

EXPERIMENT 10

File Handling and a Mini Application

Aim

To read and write files and to combine the earlier concepts in a small application in Scala.

Software and Tools Used

Scala 2.13 or 3.x, JDK 11 or above, sbt or scala-cli, IntelliJ IDEA or VS Code (Metals)

Question 10

Write Scala programs to:

- (a) Write a few lines of text to a file using `java.io.PrintWriter` (or `java.nio.file.Files`).
- (b) Read the file back using `scala.io.Source.fromFile` and print each line with its line number; close the source properly (using).
- (c) Count the number of lines, words and characters in the file.
- (d) Find the frequency of each word in the file using `groupBy` and `map`, and display the top 3 words.
- (e) Create a student record management application with a case class `Student` and a `List` that supports add, search by roll number, display all, and find the topper.
- (f) Save the student records to a CSV file and load them back into a `List[Student]`.

Date: _____

REMARKS AND SIGNATURE

Exp. No.	Date	Marks	Remarks	Teacher's Sign.
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				

Overall Remarks: _____

Signature of Lab Instructor: _____